

UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

BAR APPLICATION

Master Degree in
ICT for Internet and Multimedia

- Homework 2 -
Client-side/overall design and implementation

Group BAR

8th June 2026

Index

Bar Application

1 Our team

2 Main features

3 Backend

4 Frontend

5 Demo

6 Member contribution



1 OUR TEAM

Our Team

Bar Application

Foffano
Claudia





Iacobet
Ecaterina

Nuñez
Luis





Bruzzese
Fulvio



2 MAIN FEATURES

The problem

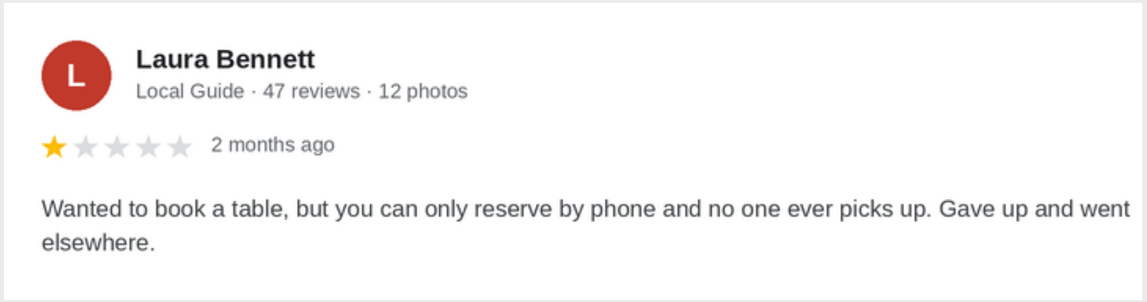
Bar Application

Sounds familiar?

The problem

Bar Application

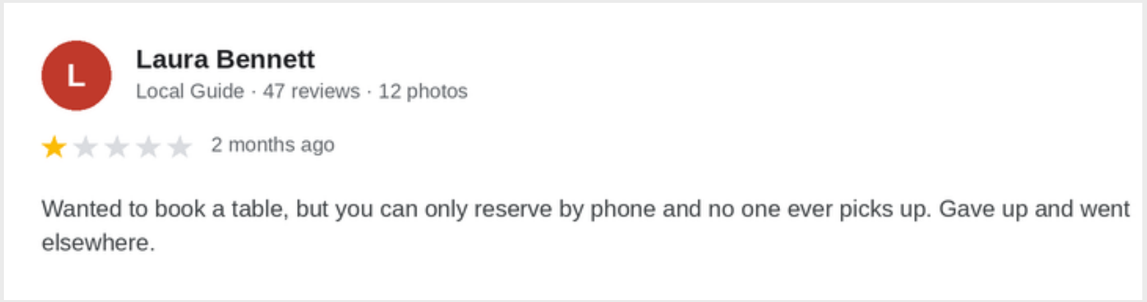
Sounds familiar?

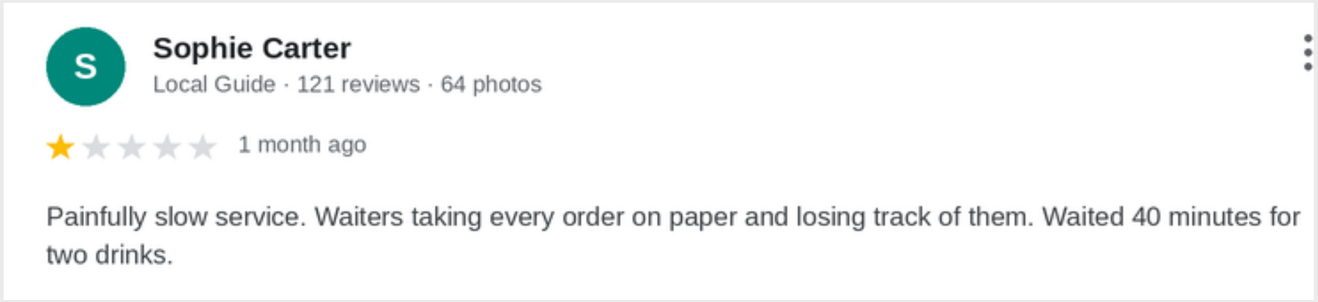


The problem

Bar Application

Sounds familiar?

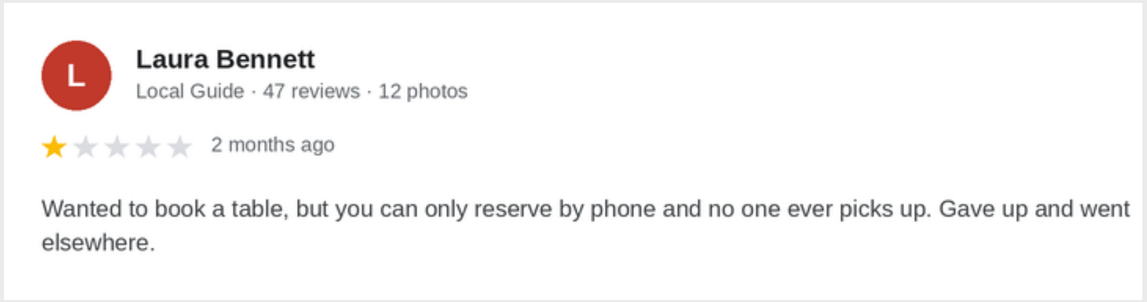


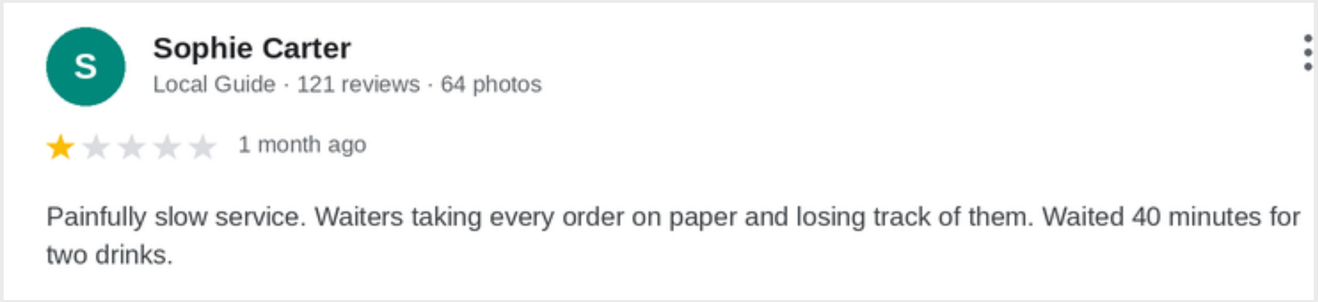


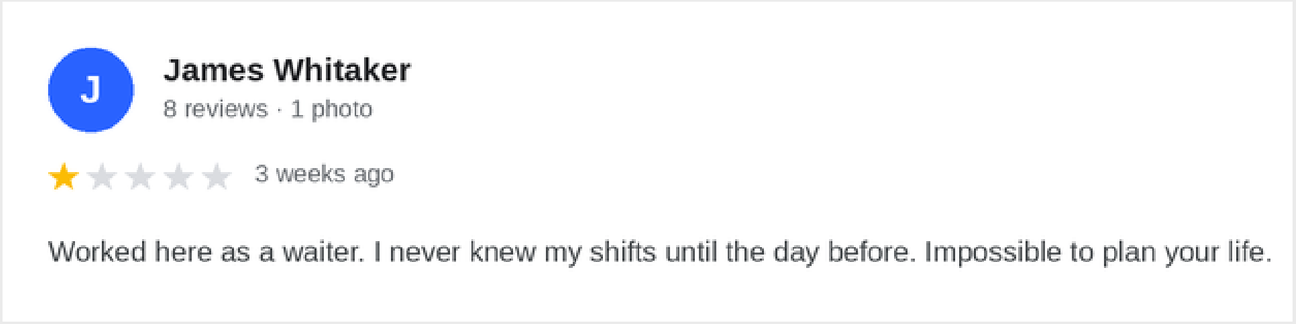
The problem

Bar Application

Sounds familiar?







The solution

Bar Application

WE TURNED EVERY COMPLAINT INTO A FEATURE

- Book Reservations
- Make orders
- Manage shifts

Login Service

Bar Application

Role-Based Application

Waiter



Customer



Main Features

Bar Application






ROLE SEPARATION

Role-based access keeps every user to the right features

LEAVE REVIEWS

Share your feedback about your experience at LoungeBar.

MANAGE SHIFTS

A simple interface to organize waiter shifts.

BOOK RESERVATIONS

Reserve your table in advance and skip the wait.

MAKE ORDERS

Start an order and pick the drinks for your table.

MANAGE SUPPLIES

Track stock in real time and never run out.

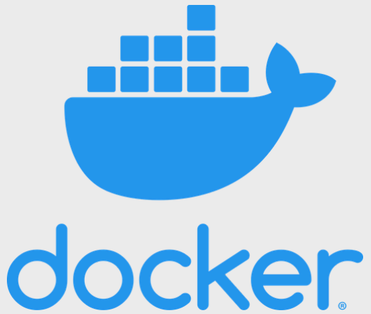


3 BACKEND

Tools

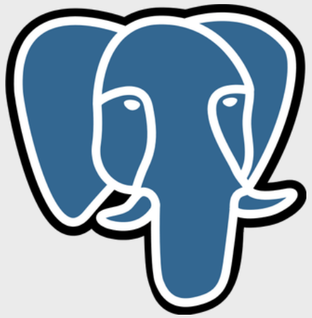
Bar Application



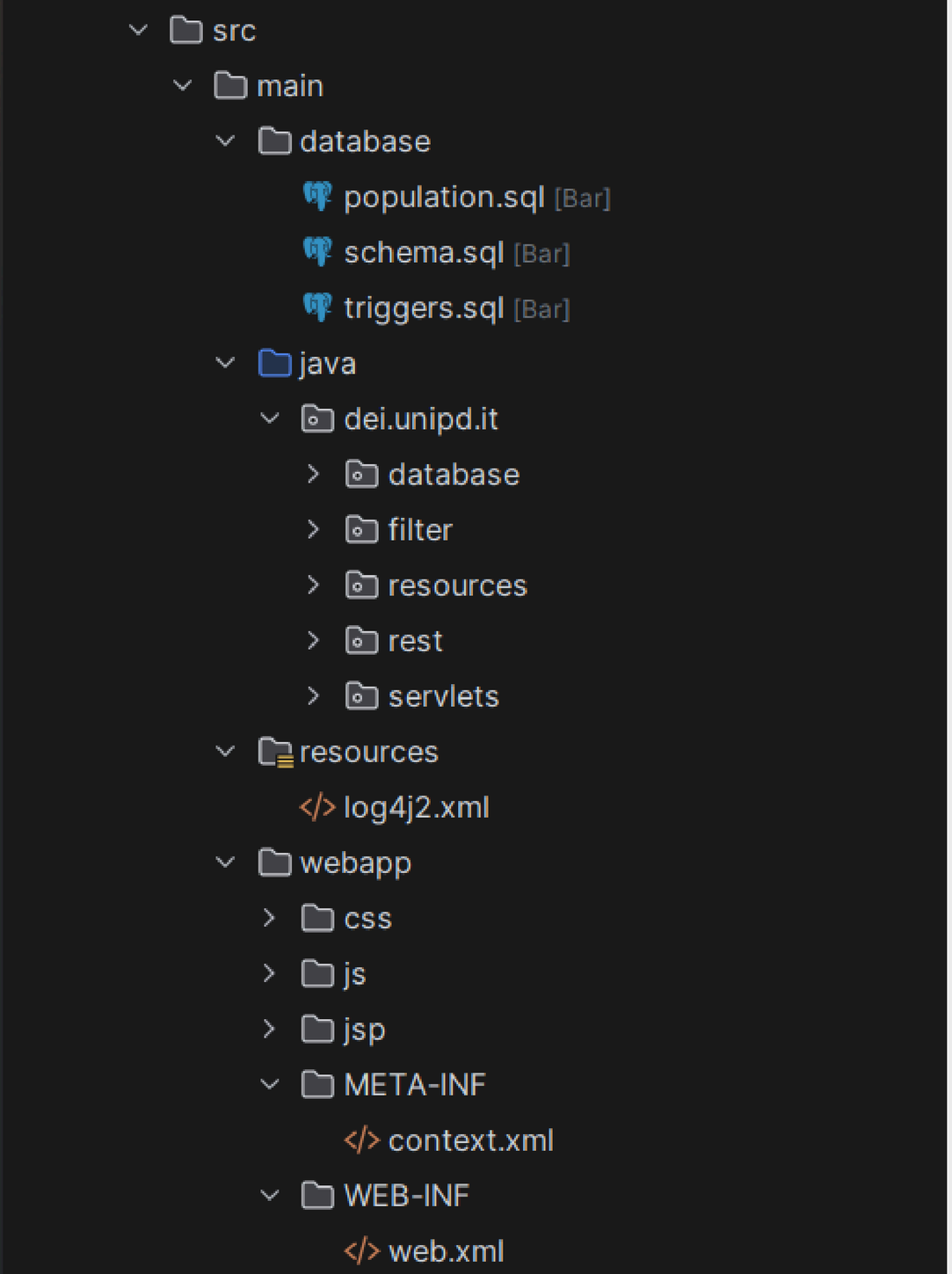








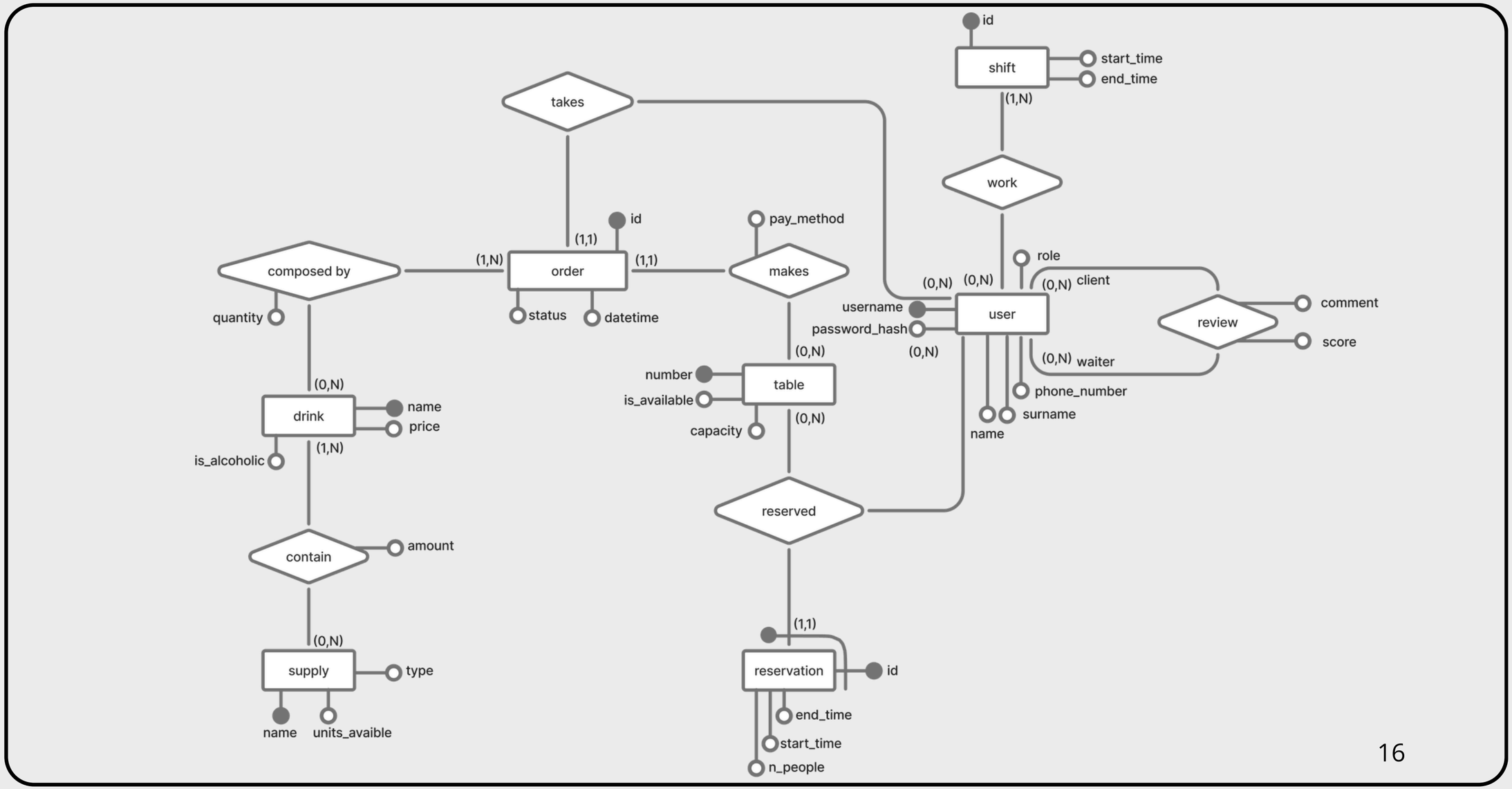




PROJECT STRUCTURE

Data layer - ER Schema

Bar Application

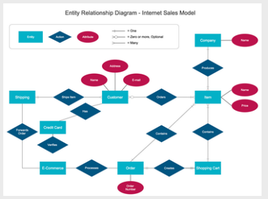


Data layer files

Bar Application

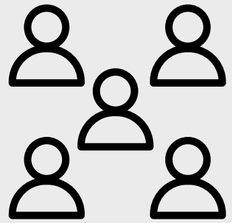


schema.sql





population.sql





triggers.sql



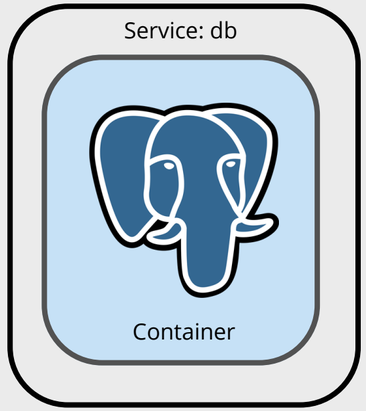
Containerization

Bar Application

```yaml
services:
  ▷Run Service
  web:
    image: tomcat:latest
    ports:
      - "8081:8080"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./target/loungebar.war:/usr/local/tomcat/webapps/loungebar.war
      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
      - ./logs:/usr/local/tomcat/webapps/my-logs
      - ./tmp/context.xml:/usr/local/tomcat/webapps.dist/manager/META-INF/context.xml
      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml

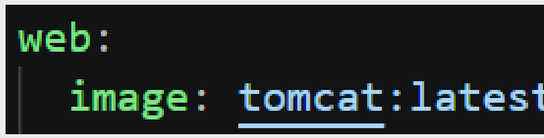
  ▷Run Service
  db:
    image: postgres
    ports:
      - "5433:5432"
    environment:
      - POSTGRES_PASSWORD=[redacted]
      - POSTGRES_USER=fskcl
    volumes:
      - ./data/db:/var/lib/postgresql
      - ./src/main/database/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql
      - ./src/main/database/population.sql:/docker-entrypoint-initdb.d/02-data.sql
      - ./src/main/database/triggers.sql:/docker-entrypoint-initdb.d/03-triggers.sql
```





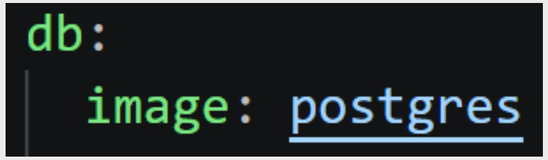
Containerization

Bar Application

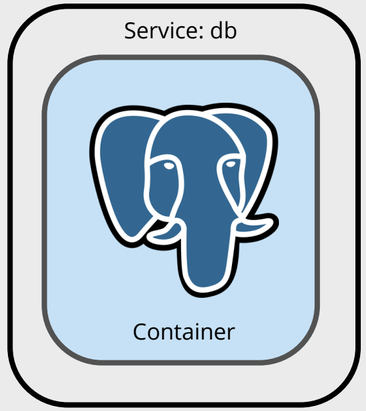












Containerization

Bar Application

```yaml
services:
  ▷Run Service
  web:
    image: tomcat:latest
    ports:
      - "8081:8080"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./target/loungebar.war:/usr/local/tomcat/webapps/loungebar.war
      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
      - ./logs:/usr/local/tomcat/webapps/my-logs
      - ./tmp/context.xml:/usr/local/tomcat/webapps.dist/manager/META-INF/context.xml
      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml
  ▷Run Service
  db:
    image: postgres
    ports:
      - "5433:5432"
    environment:
      - POSTGRES_PASSWORD=[redacted]
      - POSTGRES_USER=fskcl
    volumes:
      - ./data/db:/var/lib/postgresql
      - ./src/main/database/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql
      - ./src/main/database/population.sql:/docker-entrypoint-initdb.d/02-data.sql
      - ./src/main/database/triggers.sql:/docker-entrypoint-initdb.d/03-triggers.sql
```





Containerization

Bar Application



```yaml
volumes:
  - ./target/loungebar.war:/usr/local/tomcat/webapps/loungebar.war
  - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
  - ./logs:/usr/local/tomcat/webapps/my-logs
  - ./tmp/context.xml:/usr/local/tomcat/webapps.dist/manager/META-INF/context.xml
  - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml
```




```yaml
volumes:
  - ./data/db:/var/lib/postgresql
  - ./src/main/database/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql
  - ./src/main/database/population.sql:/docker-entrypoint-initdb.d/02-data.sql
  - ./src/main/database/triggers.sql:/docker-entrypoint-initdb.d/03-triggers.sql
```

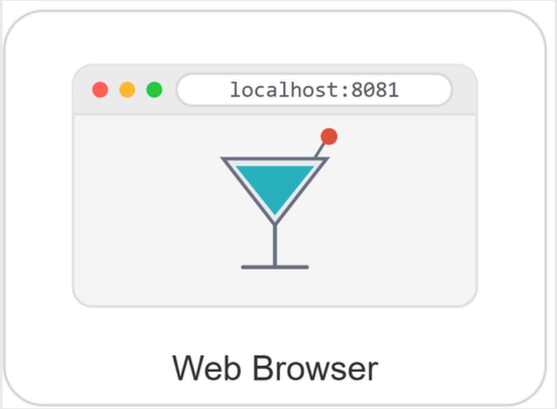


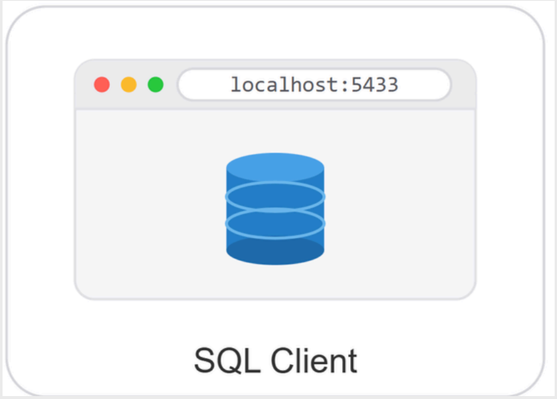
Containerization

Bar Application


```yaml
services:
  ▷Run Service
  web:
    image: tomcat:latest
    ports:
      - "8081:8080"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./target/loungebar.war:/usr/local/tomcat/webapps/loungebar.war
      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
      - ./logs:/usr/local/tomcat/webapps/my-logs
      - ./tmp/context.xml:/usr/local/tomcat/webapps.dist/manager/META-INF/context.xml
      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml

  ▷Run Service
  db:
    image: postgres
    ports:
      - "5433:5432"
    environment:
      - POSTGRES_PASSWORD=ferrothebest
      - POSTGRES_USER=fskcl
    volumes:
      - ./data/db:/var/lib/postgresql
      - ./src/main/database/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql
      - ./src/main/database/population.sql:/docker-entrypoint-initdb.d/02-data.sql
      - ./src/main/database/triggers.sql:/docker-entrypoint-initdb.d/03-triggers.sql
```



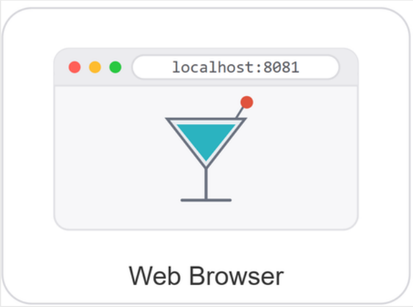


Containerization

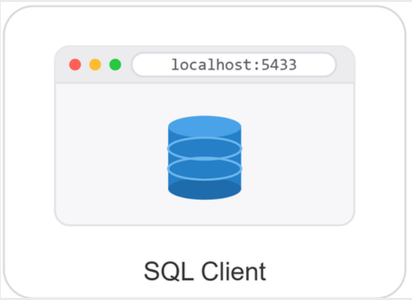
Bar Application




```
ports:
  - "8081:8080"
```

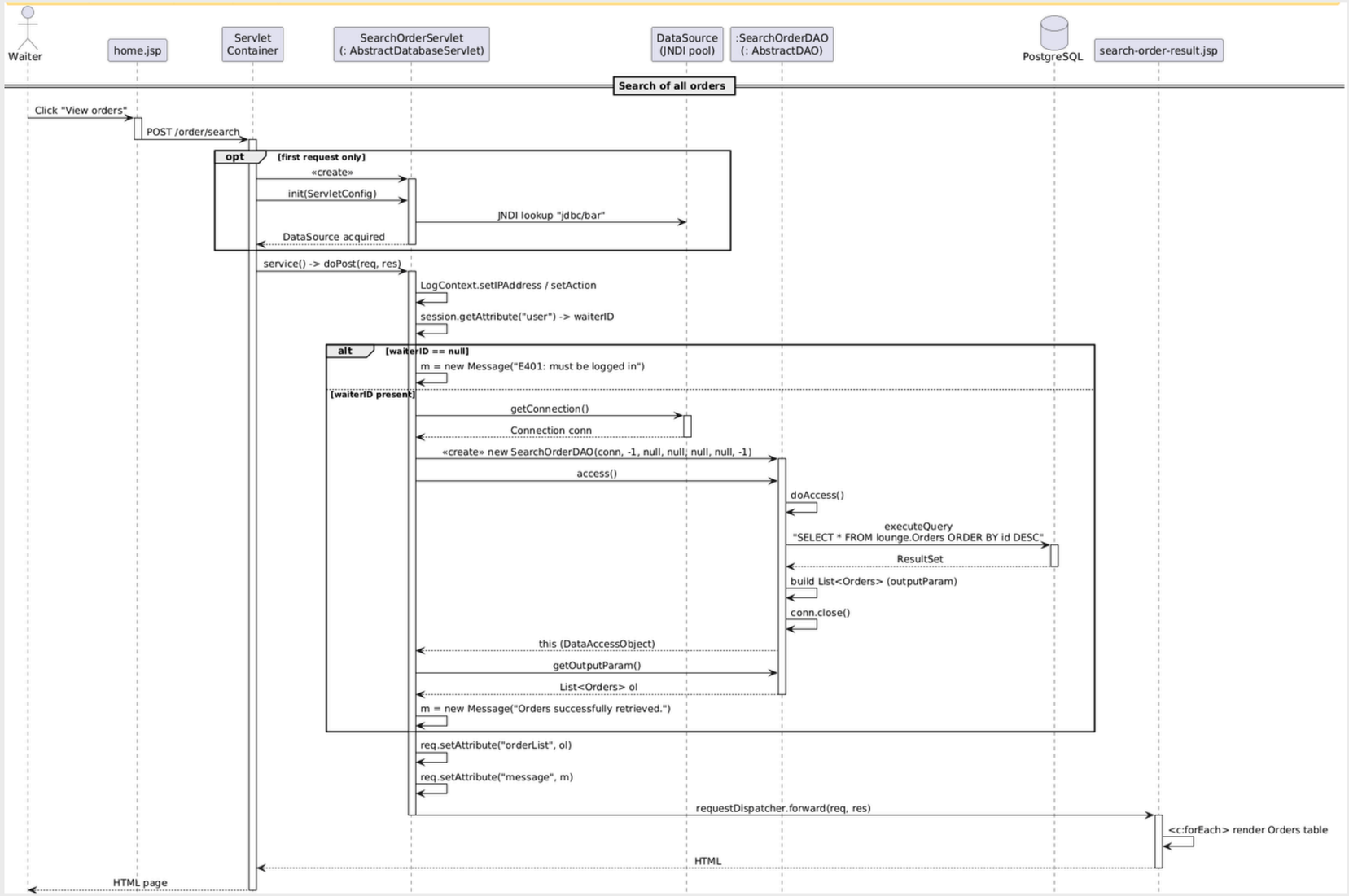



```
ports:
  - "5433:5432"
```



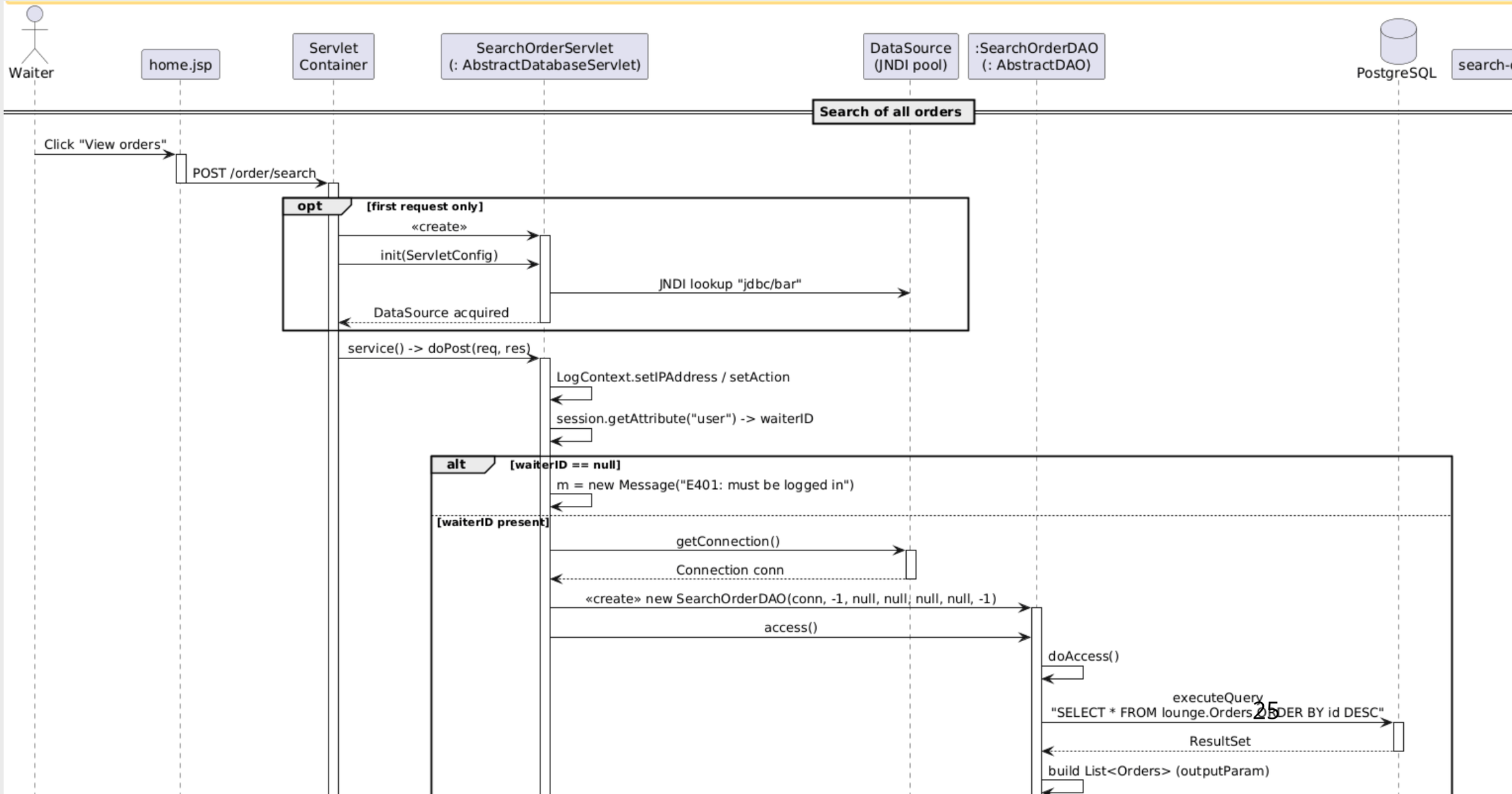
Servlets: Sequence Diagram

Bar Application



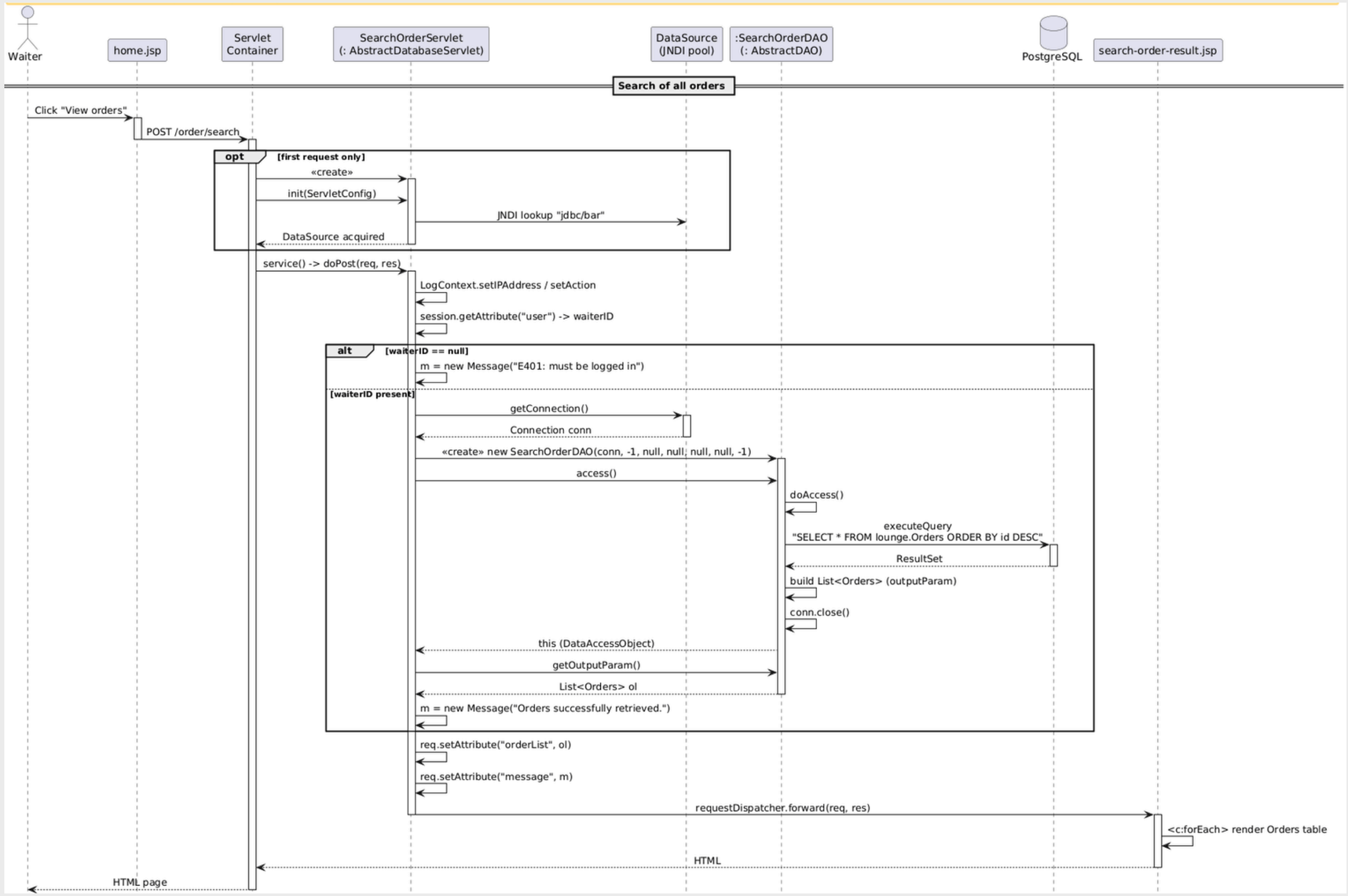
Servlets: Sequence Diagram

Bar Application



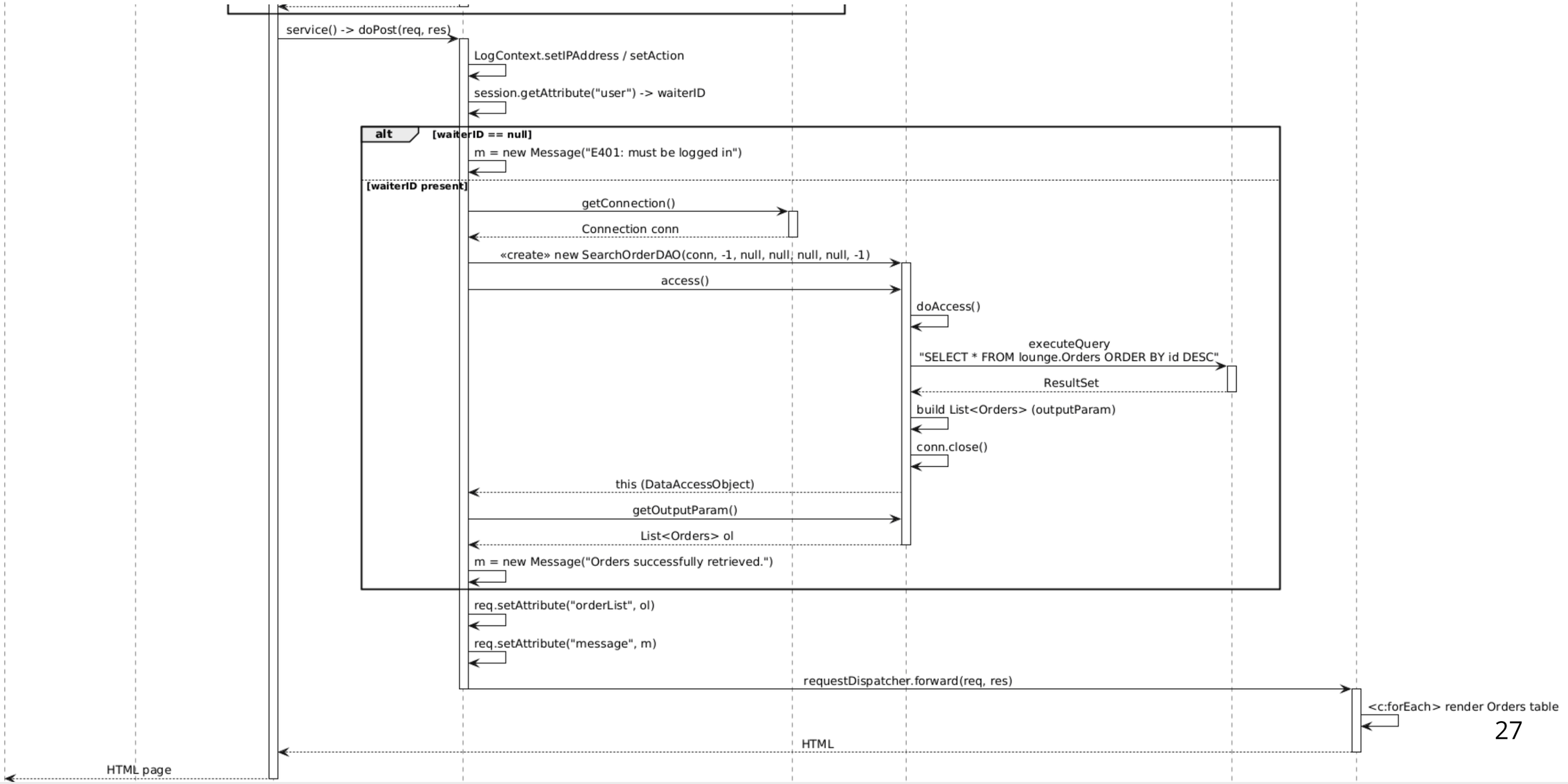
Servlets: Sequence Diagram

Bar Application



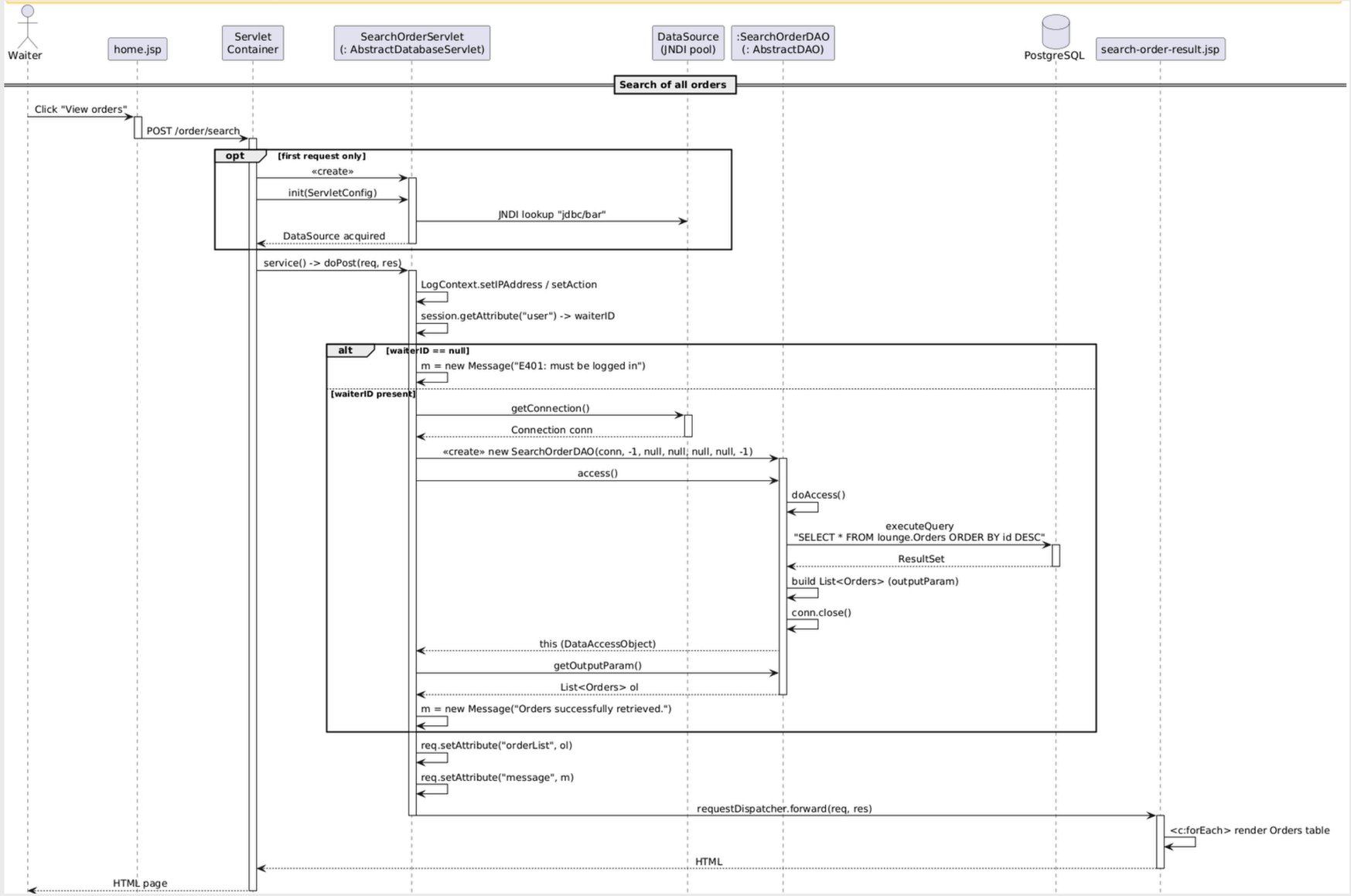
Servlets: Sequence Diagram

Bar Application



Servlets: Sequence Diagram

Bar Application



Servlets: implementation

Bar Application

Servlets: implementation

Bar Application

List of Orders

ID	DATE	WAITER NAME	STATUS	TABLE	PAY METHOD	ACTIONS
34	2026-06-04 15:30	fulvio	PENDING	1		Cash / Confirm Payment
33	2026-06-03 17:24	fulvio	PENDING	9		Cash / Confirm Payment
32	2026-06-03 11:58	fulvio	PAID	8	BANCOMAT	
31	2026-06-03 11:30	fulvio	PAID	7	BANCOMAT	
29	2026-06-03 11:28	fulvio	PAID	5	CASH	
28	2026-06-03 10:52	fulvio	PAID	3	CASH	
26	2026-06-03 10:51	fulvio	PAID	1	CASH	
25	2026-05-31 13:41	fulvio	PAID	8	CASH	
24	2026-05-29 21:30	fulvio	PAID	3	CASH	
22	2026-05-29 10:02	fulvio	PAID	1	CASH	
21	2026-05-18 13:16	fulvio	PAID	7	CASH	
19	2026-05-18 13:14	fulvio	PAID	3	CASH	
18	2026-05-12 20:47	fulvio	PAID	1	BANCOMAT	

Servlets: implementation

Bar Application

```
Handles searching for orders via POST.

Params: req – the request from the user.
        res – the response to the user.

Throws: ServletException – if a servlet error occurs.
        IOException – if an input or output error occurs.
@Override  luisnunezzc +1
public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

    LogContext.setIPAddress(req.getRemoteAddr());
    LogContext.setAction(Actions.SEARCH_ORDER);

    Message m= null;
    List<Orders> ol = null;

    try {
    // Get id of the waiter from the session
    HttpSession session = req.getSession( create: false);
    String waiterID = (session != null) ? (String) session.getAttribute( name: "user") : null;

    if (waiterID == null) {
        m = new Message( message: "You must be logged in to see your orders.", errorCode: "E401", errorDetails: "");
    } else {
        // Call DAO with the waiter ID to get the orders of that waiter
        ol = new SearchOrderDAO(getConnection(), id: -1, dateTime: null, status: null, payMethod: null, waiterID: null, tableID: -1)
                .access().getOutputParam();

        m = new Message("Orders successfully retrieved.");
        LOGGER.info("Orders successfully retrieved.");

    }

} catch (SQLException ex) {
    m = new Message( message: "Error accessing the database", errorCode: "E200", ex.getMessage());
    LOGGER.error( message: "Error searching orders", ex);
}
```

List of Orders

ID	DATE	WAITER NAME	STATUS	TABLE	PAY METHOD	ACTIONS
34	2026-06-04 15:30	fulvio	PENDING	1		Cash / Confirm Payment
33	2026-06-03 17:24	fulvio	PENDING	9		Cash / Confirm Payment
32	2026-06-03 11:58	fulvio	PAID	8	BANCOMAT	
31	2026-06-03 11:30	fulvio	PAID	7	BANCOMAT	
29	2026-06-03 11:28	fulvio	PAID	5	CASH	
28	2026-06-03 10:52	fulvio	PAID	3	CASH	
26	2026-06-03 10:51	fulvio	PAID	1	CASH	
25	2026-05-31 13:41	fulvio	PAID	8	CASH	
24	2026-05-29 21:30	fulvio	PAID	3	CASH	
22	2026-05-29 10:02	fulvio	PAID	1	CASH	
21	2026-05-18 13:16	fulvio	PAID	7	CASH	
19	2026-05-18 13:14	fulvio	PAID	3	CASH	
18	2026-05-12 20:47	fulvio	PAID	1	BANCOMAT	

Servlets: implementation

Bar Application

```
Handles searching for orders via POST.

Params: req – the request from the user.
        res – the response to the user.

Throws: ServletException – if a servlet error occurs.
        IOException – if an input or output error occurs.
```

```
@Override  luisnunezzc +1
public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

    LogContext.setIPAddress(req.getRemoteAddr());
    LogContext.setAction(Actions.SEARCH_ORDER);

    Message m= null;
    List<Orders> ol = null;

    try {
    // Get id of the waiter from the session
    HttpSession session = req.getSession( create: false);
    String waiterID = (session != null) ? (String) session.getAttribute( name: "user") : null;

    if (waiterID == null) {
        m = new Message( message: "You must be logged in to see your orders.", errorCode: "E401", errorDetails: "");
    } else {
        // Call DAO with the waiter ID to get the orders of that waiter
        ol = new SearchOrderDAO(getConnection(), id: -1, dateTime: null, status: null, payl
                .access().getOutputParam();

        m = new Message("Orders successfully retrieved.");
        LOGGER.info("Orders successfully retrieved.");

    }

} catch (SQLException ex) {
    m = new Message( message: "Error accessing the database", errorCode: "E200", ex.getMessage());
    LOGGER.error( message: "Error searching orders", ex);
}
```

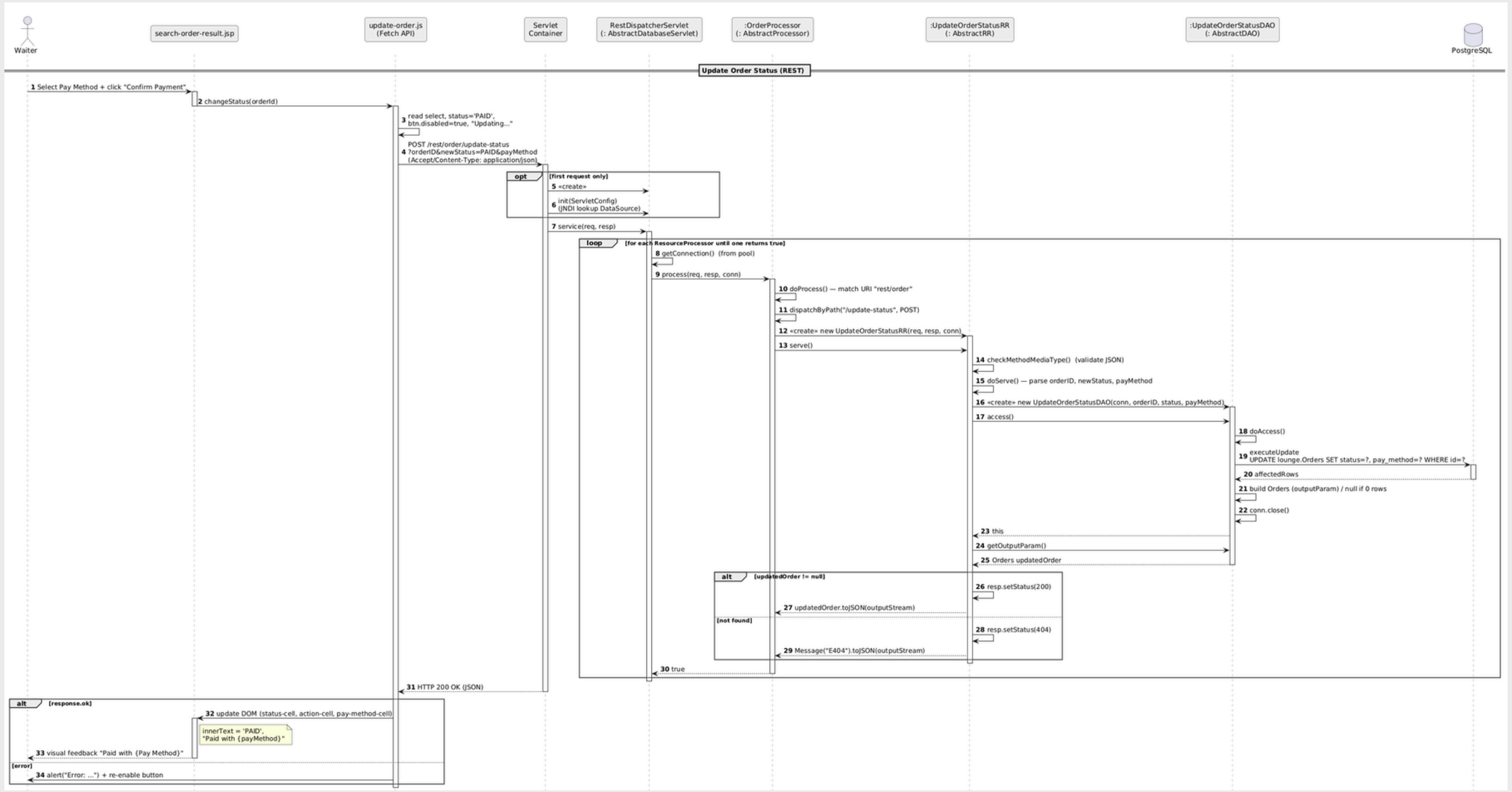
```
// Send data to jsp
req.setAttribute( name: "orderList", ol);
req.setAttribute( name: "message", m);
req.getRequestDispatcher( path: "/jsp/protected/search-order-result.jsp").forward(req, res);
```

List of Orders

WAITER NAME	STATUS	TABLE	PAY METHOD	ACTIONS
fulvio	PENDING	1		Cash / Confirm Payment
fulvio	PENDING	9		Cash / Confirm Payment
fulvio	PAID	8	BANCOMAT	
fulvio	PAID	7	BANCOMAT	
fulvio	PAID	5	CASH	
fulvio	PAID	3	CASH	
fulvio	PAID	7	CASH	
fulvio	PAID	3	CASH	

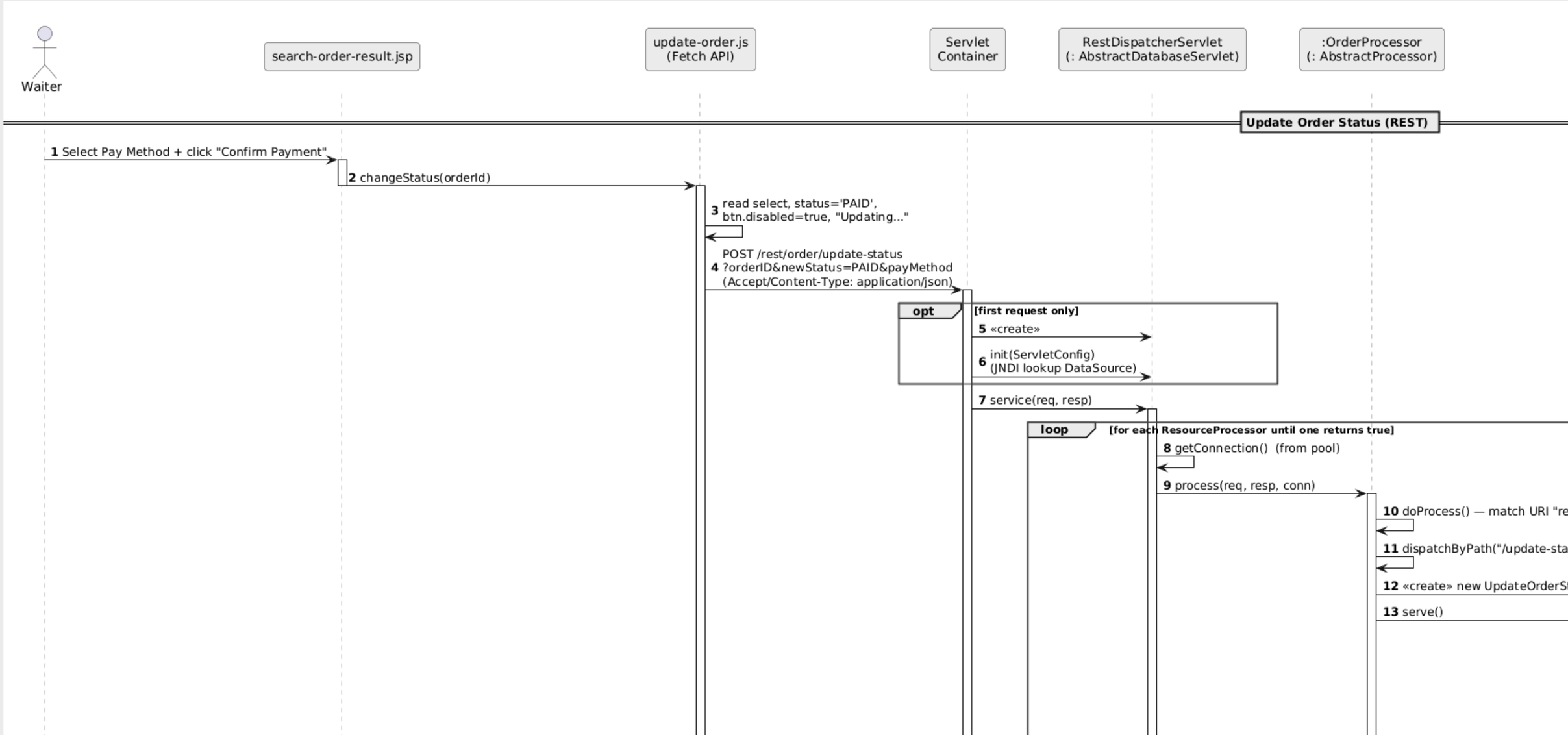
REST: Sequence Diagram

Bar Application



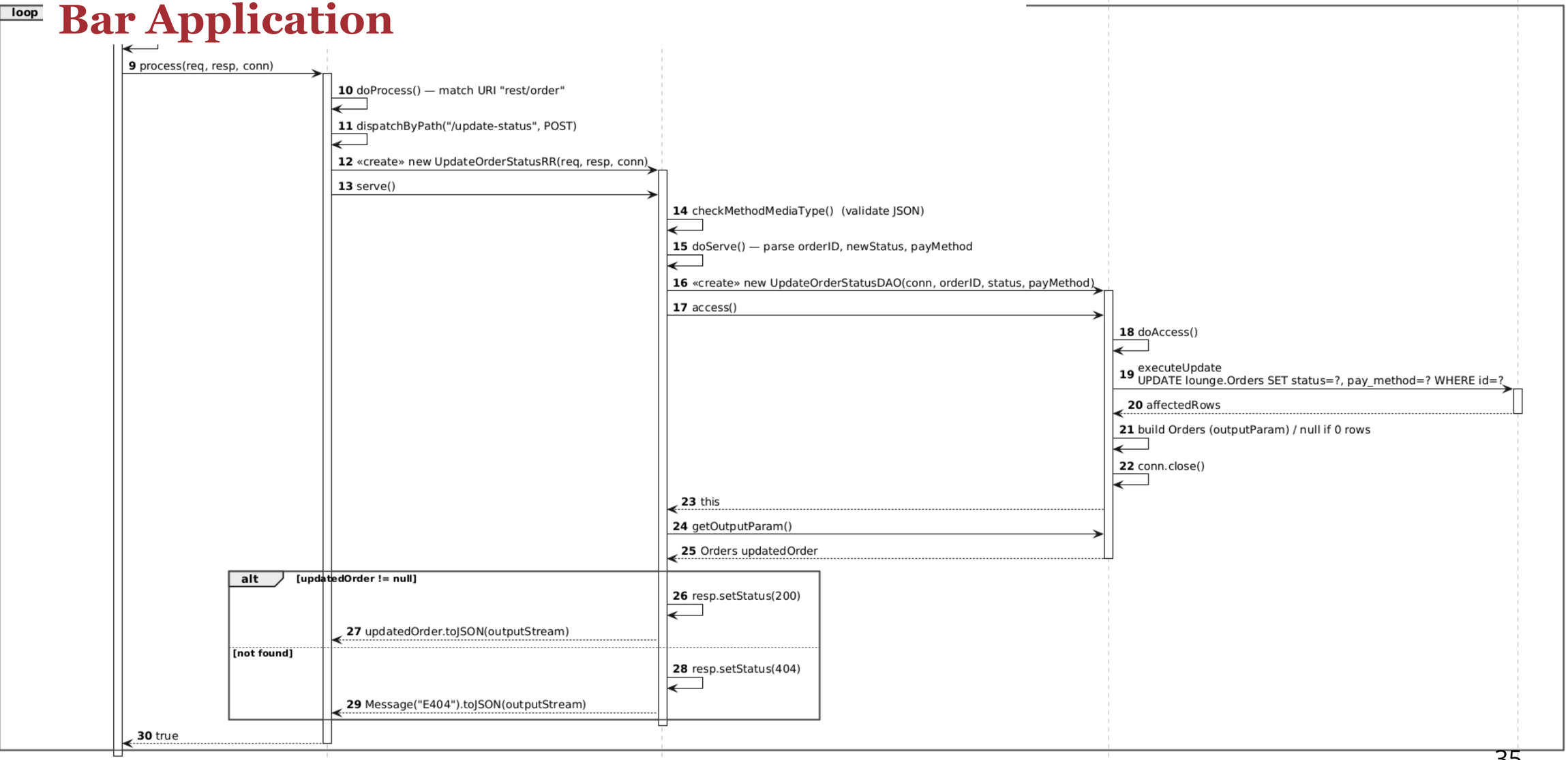
REST: Sequence Diagram

Bar Application



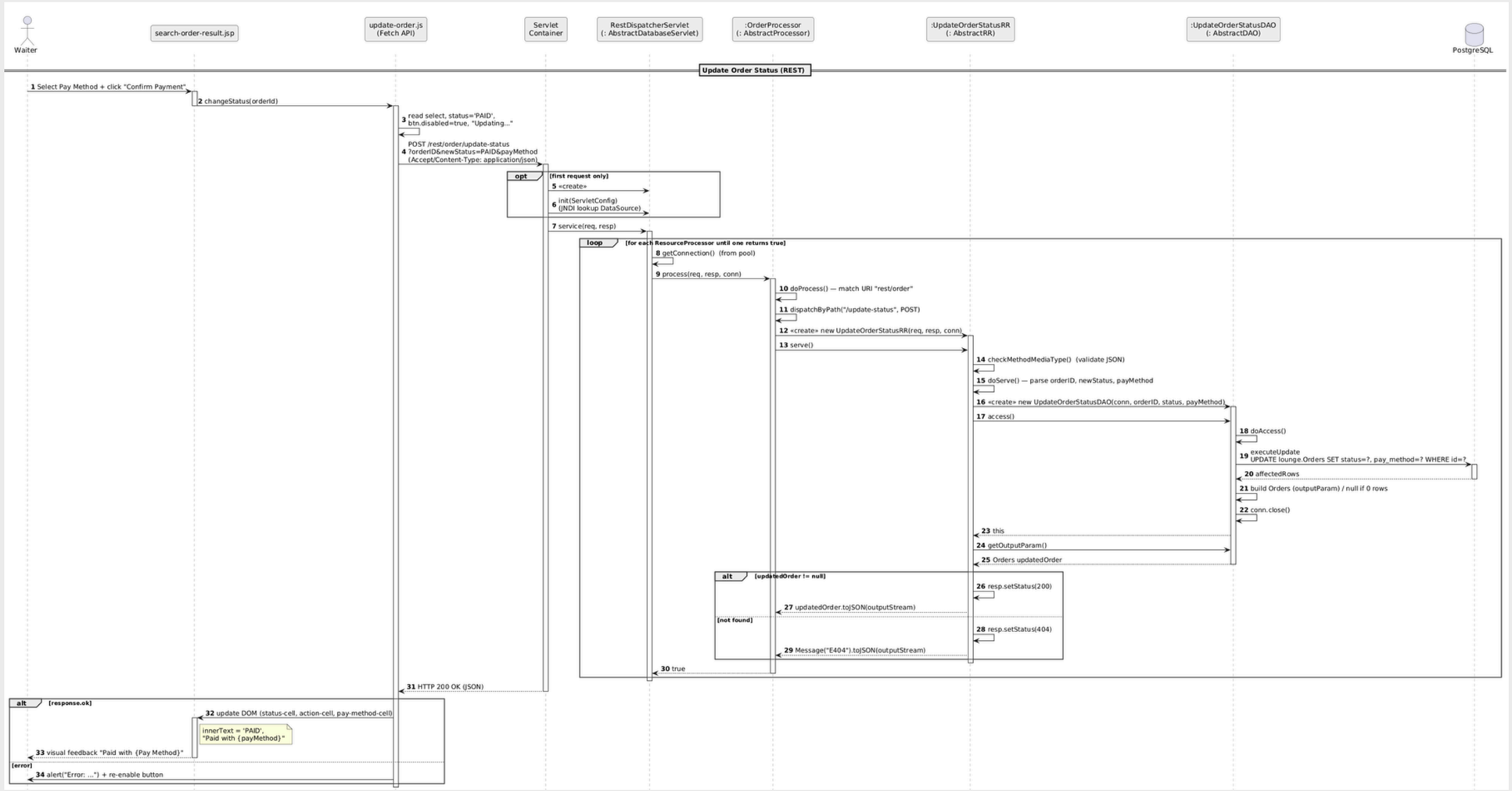
REST: Sequence Diagram

Bar Application



REST: Sequence Diagram

Bar Application



REST: implementation

Bar Application

REST: implementation

Bar Application

```java
@Override  luisnunezzc *
protected void doServe() throws IOException {
    Message message;
    Orders updatedOrder;

    try {
        // Extract parameters
        int orderID = Integer.parseInt(req.getParameter( name: "orderID"));
        OrderState newState = OrderState.valueOf(req.getParameter( name: "newStatus"));
        PayMethod newPayMethod = PayMethod.valueOf(req.getParameter( name: "payMethod"));

        LogContext.setResource(Integer.toString(orderID));

        // Execute DAO
        updatedOrder = new UpdateOrderStatusDAO(conn, orderID, newState, newPayMethod).access().getOutputParam();

        if (updatedOrder != null) {
            LOGGER.info( message: "Order {} status updated successfully", orderID);
            resp.setStatus(HttpServletResponse.SC_OK);
            // write JSON in the response
            updatedOrder.toJSON(resp.getOutputStream());
        } else {
            LOGGER.error( message: "Order not found: {}", orderID);
            message = new Message( message: "Cannot update: order not found.",  errorCode: "E404",  errorDetails: null);
            resp.setStatus(HttpServletResponse.SC_NOT_FOUND);
            message.toJSON(resp.getOutputStream());
        }

    } catch (NullPointerException | IllegalArgumentException e) {
        LOGGER.warn( message: "Invalid input for update status.", e);
        message = new Message( message: "Invalid input parameters.",  errorCode: "E400", e.getMessage());
        resp.setStatus(HttpServletResponse.SC_BAD_REQUEST);
        message.toJSON(resp.getOutputStream());

    } catch (SQLException ex) {
        LOGGER.error( message: "Database error while updating order status.", ex);
        message = new Message( message: "Database error.",  errorCode: "E500", ex.getMessage());
        resp.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
        message.toJSON(resp.getOutputStream());
    }
}
```

REST: implementation

Bar Application




```java
@Override  luisnunezzc *
protected void doServe() throws IOException {
    Message message;
    Orders updatedOrder;

    try {
        // Extract parameters
        int orderID = Integer.parseInt(req.getParameter( name: "orderID"));
        OrderState newState = OrderState.valueOf(req.getParameter( name: "newStatus"));
        PayMethod newPayMethod = PayMethod.valueOf(req.getParameter( name: "payMethod"));

        LogContext.setResource(Integer.toString(orderID));

        // Execute DAO
        updatedOrder = new UpdateOrderStatusDAO(conn, orderID, newState, newPayMethod).access().getOutputParam();

        if (updatedOrder != null) {
            LOGGER.info( message: "Order {} status updated successfully", orderID);
            resp.setStatus(HttpServletResponse.SC_OK);
            // write JSON in the response
            updatedOrder.toJSON(resp.getOutputStream());
        } else {
            LOGGER.error( message: "Order not found: {}", orderID);
            message = new Message( message: "Cannot update: order not found.",  errorCode: "E404",  errorDetails: null);
            resp.setStatus(HttpServletResponse.SC_NOT_FOUND);
            message.toJSON(resp.getOutputStream());
        }

    } catch (NullPointerException | IllegalArgumentException e) {
        LOGGER.warn( message: "Invalid input for update status.", e);
        message = new Message( message: "Invalid input parameters.",  errorCode: "E400", e.getMessage());
        resp.setStatus(HttpServletResponse.SC_BAD_REQUEST);
        message.toJSON(resp.getOutputStream());

    } catch (SQLException ex) {
        LOGGER.error( message: "Database error while updating order status.", ex);
        message = new Message( message: "Database error.",  errorCode: "E500", ex.getMessage());
        resp.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
        message.toJSON(resp.getOutputStream());
    }
}
```

REST: implementation

Bar Application




```java
// Define URL: /rest/order
private final static String SEGMENT = "order";  2 usages

@Override  1 usage  luisnunezzc
protected boolean doProcess(HttpServletRequest req, HttpServletResponse resp, Connection conn) throws Exception {

    final String uri = req.getRequestURI();
    // Search in the URL
    final int segmentIndex = uri.lastIndexOf( str: "rest/" + SEGMENT);

    if (segmentIndex < 0) {
        return false;
    }

    // Extract what after the url
    final String subPath = uri.substring( beginIndex: segmentIndex + "rest/".length() + SEGMENT.length());

    dispatchByPath(subPath, req, resp, conn);
    return true;
}
```

REST: implementation

Bar Application




```java
// Define URL: /rest/order
private final static String SEGMENT = "order";  2 usages

@Override  1 usage  luisnunezzc
protected boolean doProcess(HttpServletRequest req, HttpServletResponse resp, Connection conn) throws Exception {

    final String uri = req.getRequestURI();
    // Search in the URL
    final int segmentIndex = uri.lastIndexOf( str: "rest/" + SEGMENT);

    if (segmentIndex < 0) {
        return false;
    }

    // Extract what after the url
    final String subPath = uri.substring( beginIndex: segmentIndex + "rest/".length() + SEGMENT.length());

    dispatchByPath(subPath, req, resp, conn);
    return true;
}
```

```java
private void dispatchByPath(String subPath, HttpServletRequest req, HttpServletResponse resp, Connection conn) throws IOException {

    final String method = req.getMethod();


    if (subPath.equals("/update-status")) {

        switch (method) {
            case "POST" -> new UpdateOrderStatusRR(req, resp, conn).serve();
            default     -> methodNotAllowed(resp, method,  resource: "/order/update-status");
        }

    } else {

        resp.setStatus(HttpServletResponse.SC_NOT_FOUND);
        LOGGER.warn( message: "Unknown subpath for OrderProcessor: %s", subPath);
    }
}
```

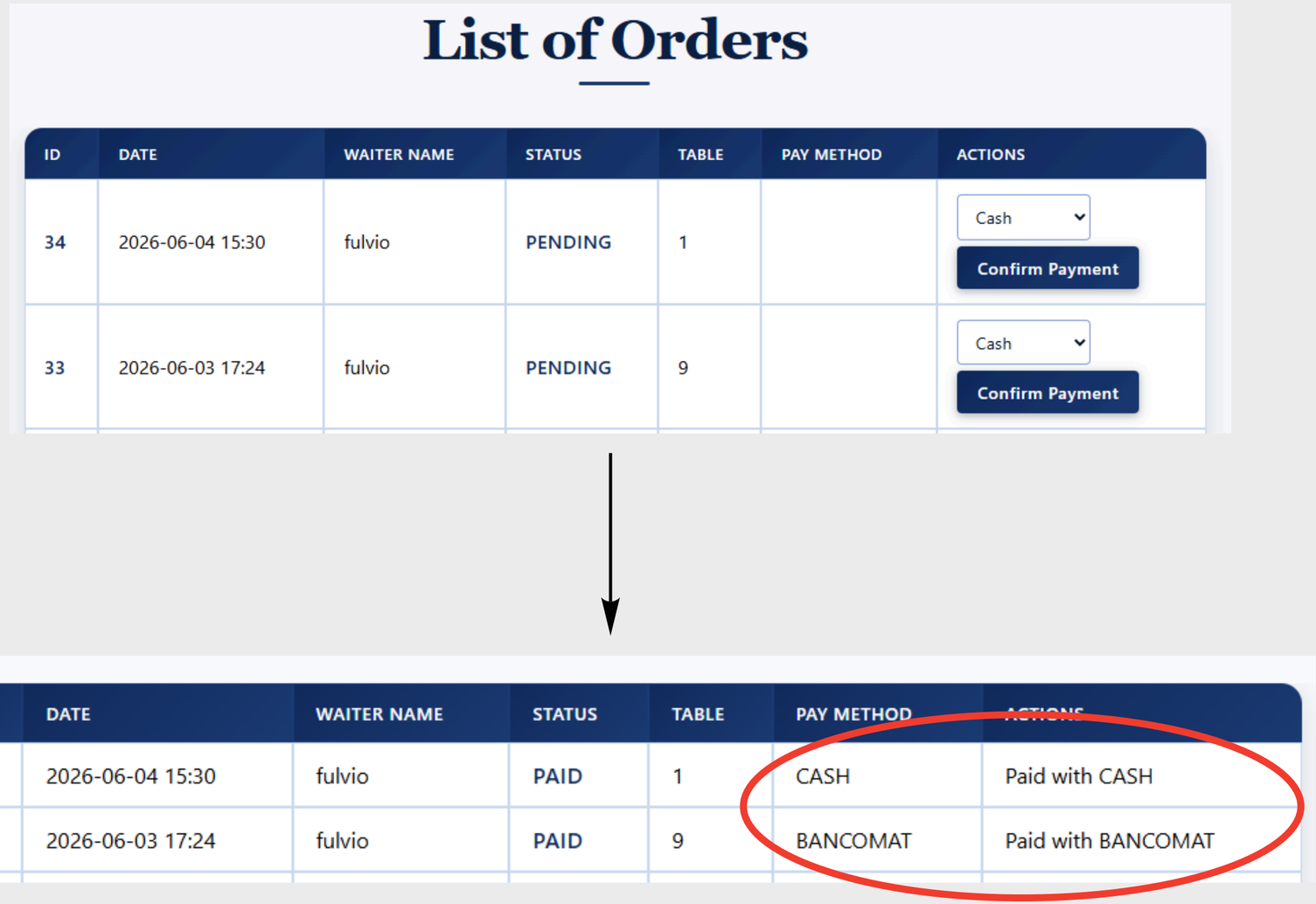
REST: implementation

Bar Application



REST: implementation

Bar Application





4 FRONTEND

External Style Sheets

Bar Application

JSP file

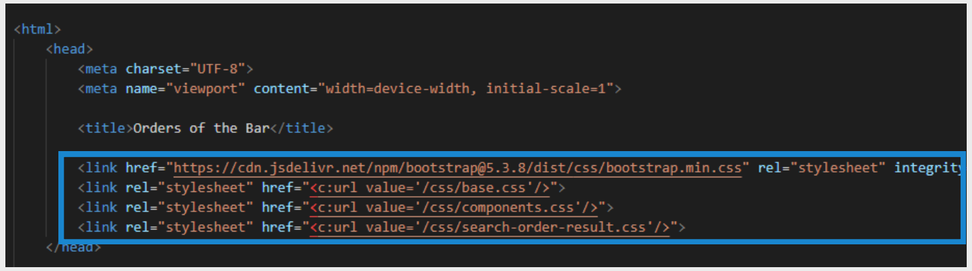
```
<html>
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">

        <title>Orders of the Bar</title>

        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet" integrity
        <link rel="stylesheet" href="<c:url value='/css/base.css'/>">
        <link rel="stylesheet" href="<c:url value='/css/components.css'/>">
        <link rel="stylesheet" href="<c:url value='/css/search-order-result.css'/>">
    </head>
```

External Style Sheets

Bar Application




```
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet" integrity="
    <link rel="stylesheet" href="<c:url value='/css/base.css'/>">
    <link rel="stylesheet" href="<c:url value='/css/components.css'/>">
    <link rel="stylesheet" href="<c:url value='/css/search-order-result.css'/>">
</head>
```

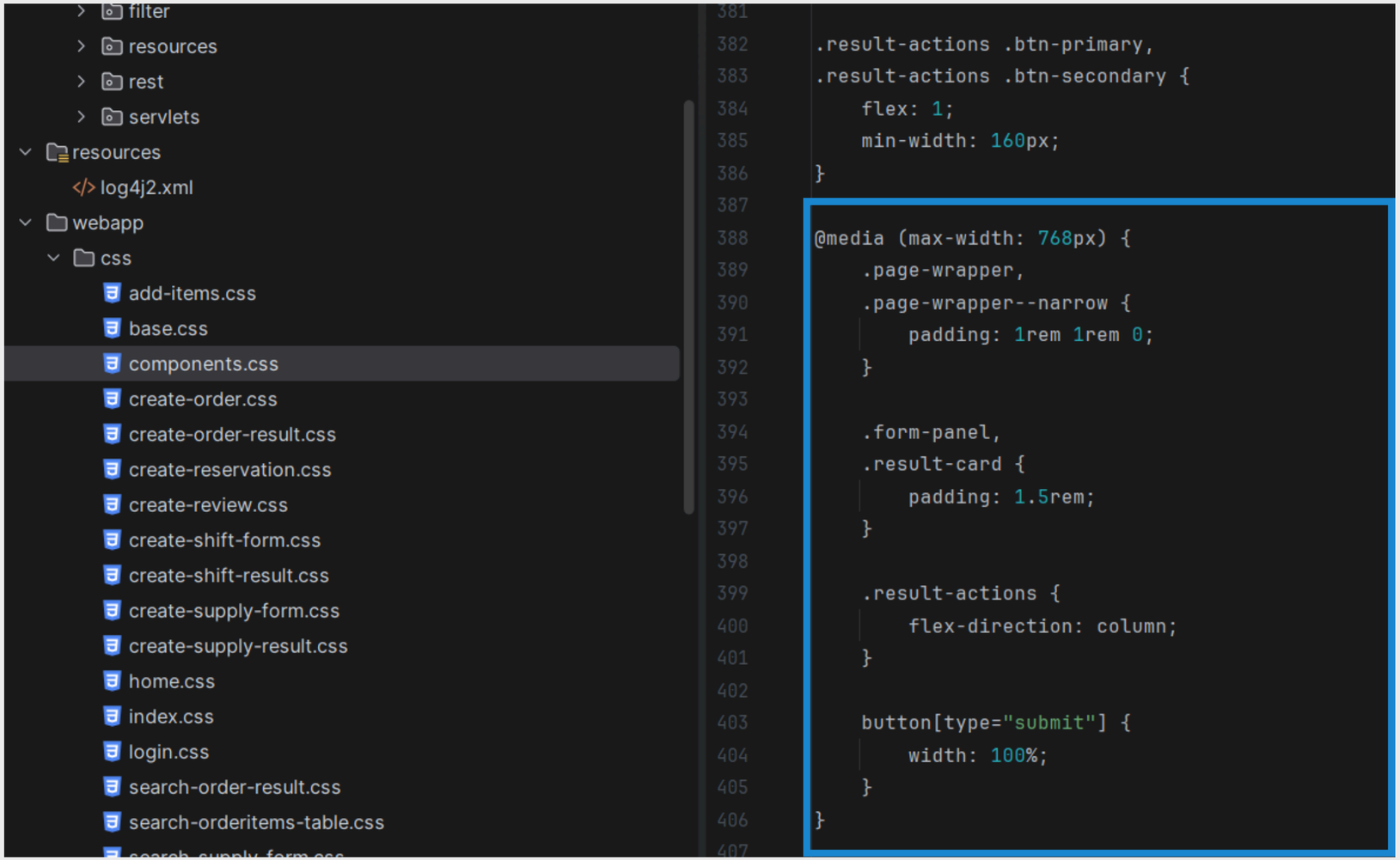
- Independency;
- Reusability;
- Easier maintenance;
- Code-separation;
- Reduce repetition;
- Consistency;

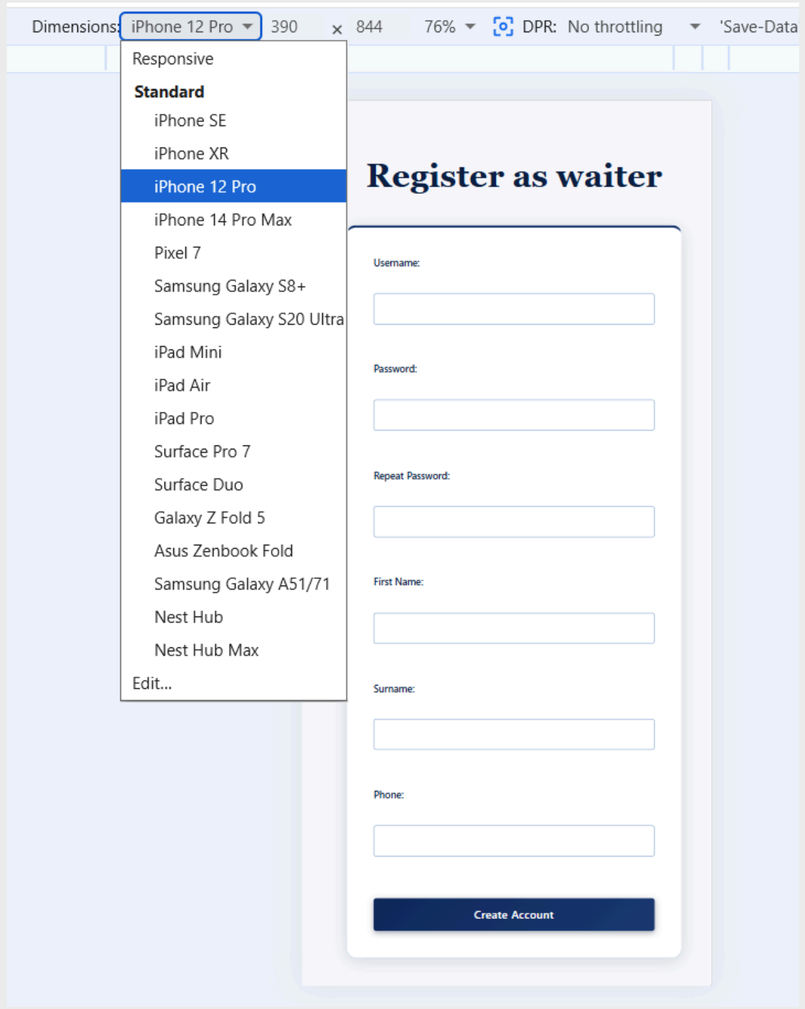
Responsiveness

Bar Application



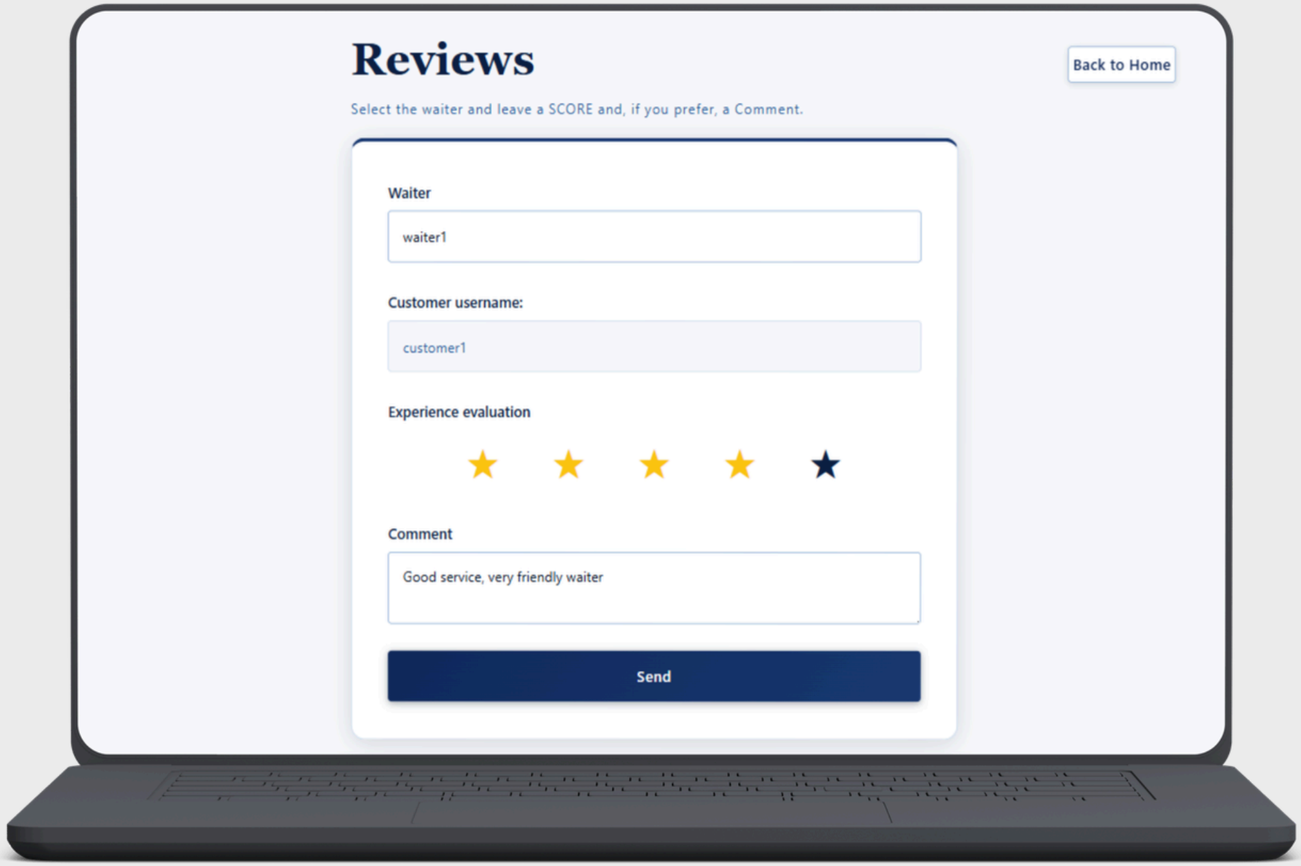
Media Query





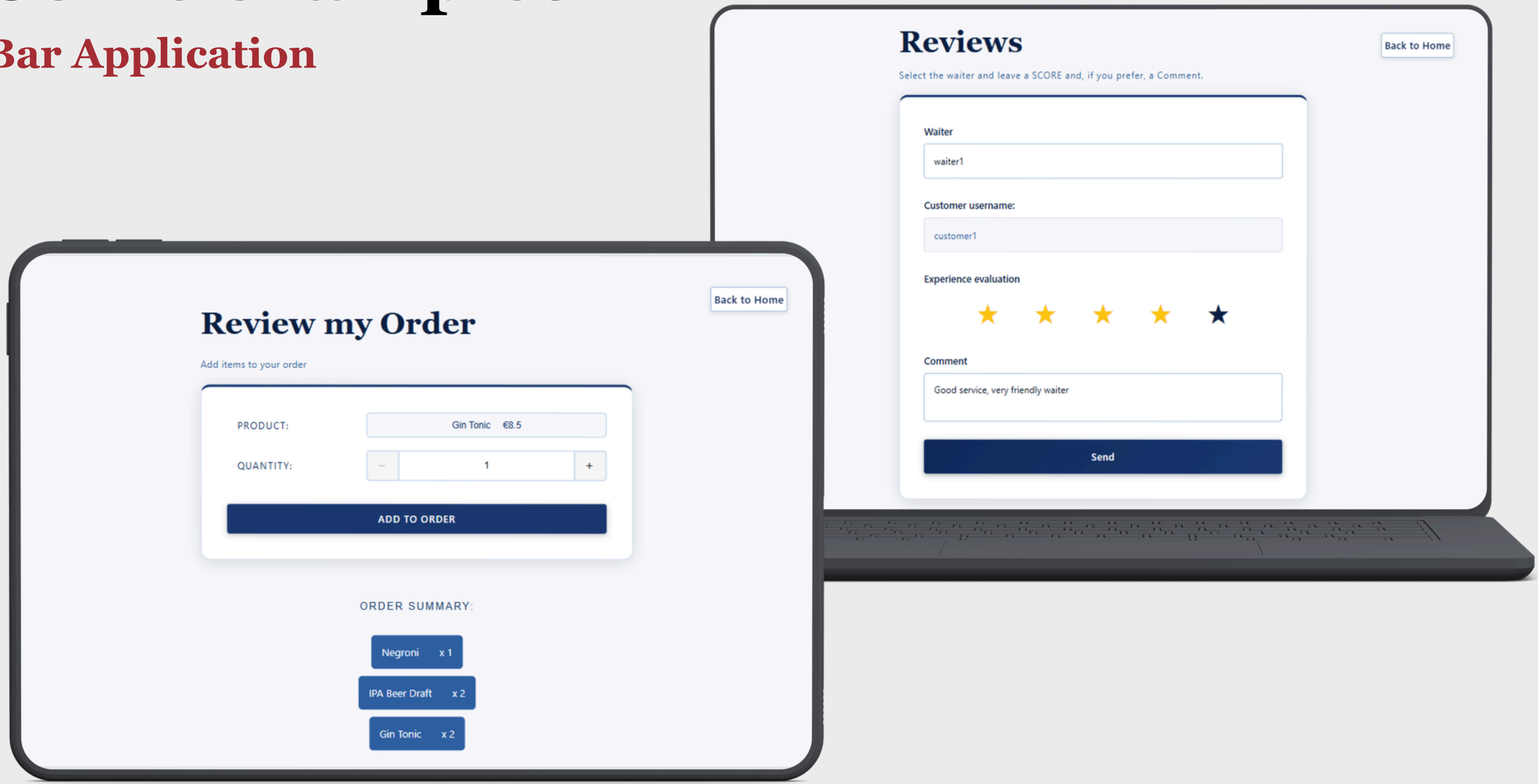
Some examples

Bar Application



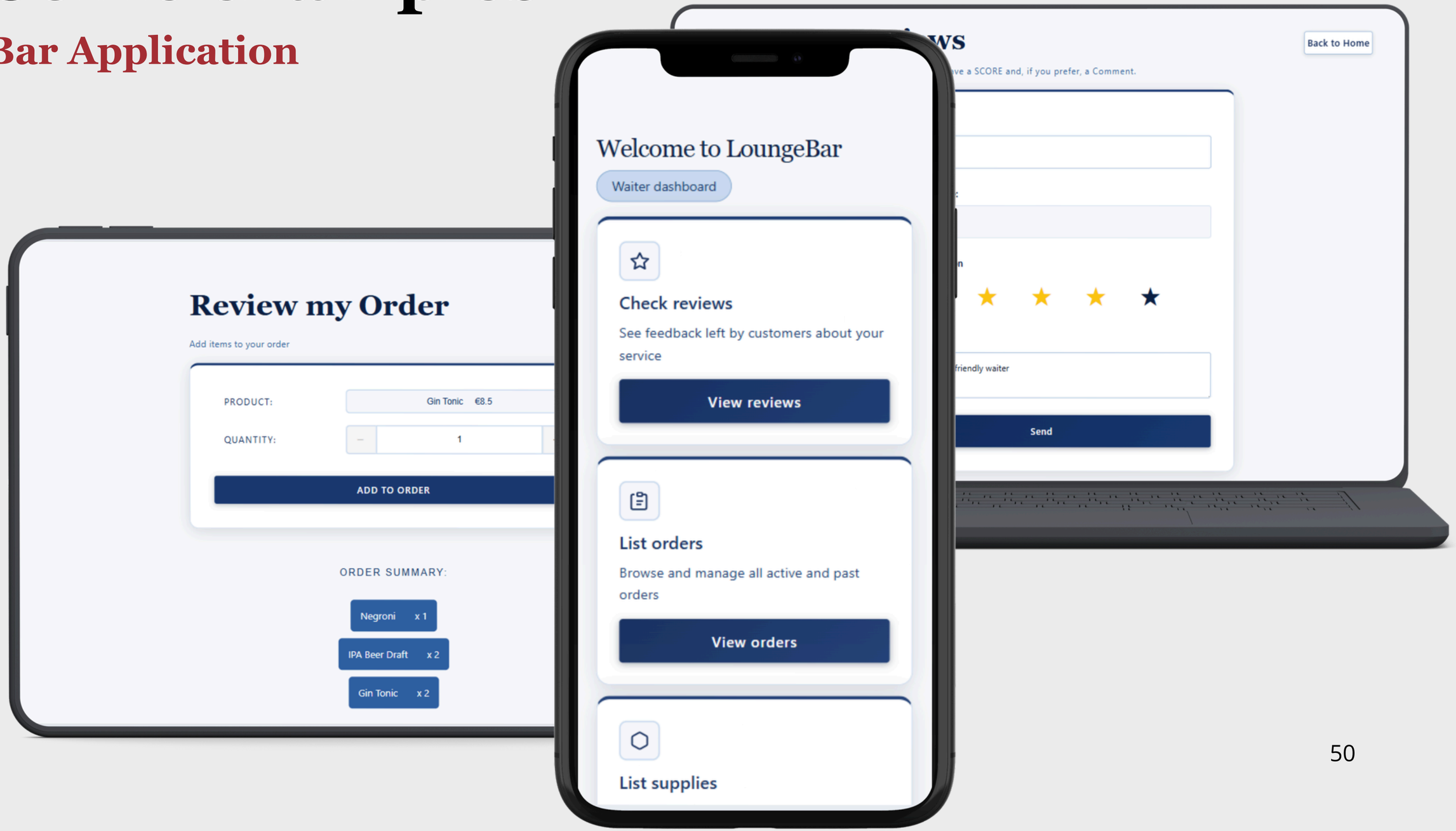
Some examples

Bar Application



Some examples

Bar Application





5 DEMO



6 MEMBER CONTRIBUTIONS

Contribution

Bar Application



Bruzzese Fulvio

- RESERVATION MANAGEMENT SERVICE
- REVIEW MANAGEMENT SERVICE
- SUPPLY MANAGEMENT SERVICE
- ORDER ITEMS SERVICE



Foffano Claudia

- LANDING PAGE
- LOGIN SERVICE
- SIGN UP SERVICE
- CREATE SUPPLY MANAGEMENT SERVICE



Iacobet Ecaterina

- SHIFT MANAGEMENT SERVICE
- HOME PAGE REPRESENTATION
- SUPPLY MANAGEMENT SERVICE
- COMMON COMPONENTS



Nuñez Luis

- ORDER MANAGEMENT SERVICE
- WAITER REVIEW SERVICE
- SEARCH ORDER SERVICE
- WAITER SHIFTS SERVICE

Thank you for your attention!